BUILD REPORT · NOVEMBER 2026

System Portal

A multi-page site rebuilt as a cinematic Solo Leveling System interface — eleven pages, one shared design system, live frame animation, and working client-side state.

11	4	6	270	8090
Pages	Chapters	Modules	Frames	Port

What this is

System Portal is a gamified progression dashboard: it turns everyday habits into timed quests, awards XP, gold and ability points, and records how you change over time. The previous version was a single long page built on a dark, neon “system window” aesthetic. This rebuild keeps every feature but restructures it as a real website — one page per chapter, one page per module — and re-scores it to the Solo Leveling anime while keeping the layout grammar from *antigravity.google-design.md*.

Design direction

Antigravity supplied the structural grammar — pill controls, 8px cards, hairline borders and airy spacing. The palette and motion were re-scored to the Solo Leveling anime, so the same skeleton now reads as a System window:

- **Near-black canvas.** A deep `#05060c` base with violet and cyan radial washes, a fixed vignette and fine scanlines.
- **Violet hologram.** `#8b5cf6` carries the anime System UI: glows, notification panels, stat bars and focus rings. Cyan `#38bdf8` is the secondary signal.
- **Pill and 8px geometry.** Buttons stay fully rounded; cards, inputs and modules use 8px radii with translucent hairline borders.
- **Angular display type.** Chakra Petch for headings, Inter for body copy and JetBrains Mono for labels and frame counters.
- **Light as material.** Panel glows, HUD corner brackets, animated tickers and a cursor-like progress beam replace the flat white surfaces.

One design system, eleven pages

Every page loads the same two stylesheets and the same script. Only the page body differs, so the site cannot drift out of alignment as it grows.

Page map

Page	Files	What it does
Home	`index.html`	Autoplaying 90-frame opening, boot curtain, and indexes for every chapter and module.
Chapters ×4	`awakening · extraction` `arise · army`	One full-bleed autoplay stage per chapter, with HUD corners, frame counter, progress bar and play/pause. Each owns a slice of the 270 frames.
Dashboard ×6	`status · quests · vision` `advisor · progress · inventory`	One tool per page: stat allocation, timed quests, webcam check-in, advisor chat, transformation feed and dimensional inventory.

IMPLEMENTATION

How it is built

A static, framework-free build: HTML for structure, two CSS layers for the design system, and one JavaScript module for shared behaviour. No build step, no dependencies to install.

1 · Architecture

The site is deliberately DRY without a framework. Navigation and footer live in a single array inside `scripts/app.js` and are injected into an empty `<header id="site-header">` and `<footer id="site-footer">` on every page at `DOMContentLoaded`. Editing the nav once updates all eleven pages, and the active link is derived from the page's own `data-page` attribute, so highlighting can never desync.

Page logic uses the same dispatch idea: `Pages[page]()` is looked up from `document.body.dataset.page` and called in a try/catch, so a fault in one page cannot break the shared shell on another.

2 · The design system, in two layers

File	Contents
`styles/design-system.css`	Tokens (palette, type scale, spacing, radii, motion), reset, typography classes, header/nav/dropdown, buttons, cards, chips, forms, footer, responsive rules.
`styles/pages.css`	Composition only: hero and chapter stage, feature grid, module layout, gauges, quest rows, chat, transformation cards, inventory, modal, toast, plus the motion layer — boot curtain, ticker, notification window, player controls, reveals.

3 · Shared state

A small `Store` module reads and writes one JSON object to `localStorage` under `system_portal_state_v2` — level, rank, gold, fatigue, available ability points, EXP, the five stats, the quest list, reminders and transformation entries. Every page reads from the same object, so completing a quest on the Quests page immediately changes the level and gold shown on Status. A **Reset progress** control restores the seeded default state.

4 · Motion and animation

The 270 supplied frames are reused as a real, autoplaying sequence rather than a scroll-driven scrubber. A single reusable player preloads a frame slice, paints it to a canvas with device-pixel-ratio scaling and cover-fit cropping, advances at a fixed frame rate, loops, and pauses itself whenever it leaves the viewport. It also reports load progress back to the page — that is what drives the boot meter.

- **Home hero** plays the opening 90 frames at 20fps (about four and a half seconds) behind the headline.
- **Chapters** each own a contiguous slice and play at a cinematic 15fps, with an injected play/pause button, a live progress bar and a frame counter.
- **Boot sequence.** On the first visit the site opens behind a System initialisation curtain with a rotating sigil, a caching meter and a scan beam; it lifts once frames are ready.
- **Ambient layer.** A fixed canvas drifts violet and cyan mana motes across every page, with no layout cost and automatic pause when the tab is hidden.
- **Interface motion.** Scroll reveals with stagger, count-up hero stats, a rotating system signal line, a marquee status ticker, hover glows, a reading-progress beam and a live nav clock.
- **Accessibility.** Every animation is disabled or reduced under `prefers-reduced-motion`, and reveals have a timed fallback so content can never stay hidden.

5 · Module behaviour

- **Status** derives max HP, max MP and EXP percentage from stats and level, so spending a point visibly moves the gauges.
- **Quests** ticks every second, turns its timer red inside the final five minutes, awards XP and gold on completion, rolls over levels, and applies the level/fatigue penalty on expiry.
- **Vision** requests a webcam stream, captures a frame to an offscreen canvas, and then presents a simulated posture report; the reminder panel runs real countdowns and can fire a browser notification.
- **Advisor** answers from a local keyword knowledge base covering leveling, stat builds, penalties, workouts and extraction — no network call, no key.
- **Progress** reads two images with `FileReader` and stores them as data URLs only after an explicit consent checkbox; nothing leaves the browser.
- **Inventory** renders equipped and empty slots plus the skill arsenal, with click feedback routed through the shared toast.

RUNNING & VERIFYING

Open it locally

The site is served as static files. Any static server works; this build runs on port 8090 because port 8000 is already occupied on this machine by an unrelated service.

Start the server

```
cd "C:\Users\alkes\OneDrive\Desktop\system google"
python -m http.server 8090
```

Then open **http://localhost:8090**. The site is already running on that address if the server was left in the background. No install step and no build step are required — the browser loads plain HTML, CSS and JS.

What was checked

A validation script (`verify.py`) runs three classes of check and passes all of them:

- **Syntax.** `scripts/app.js` is parsed with `node --check`.
- **Structure.** All eleven pages contain the header/footer placeholders, both stylesheets, the shared script, the toast hub and a `data-page` attribute.
- **Contract.** Every element ID and class that the JavaScript queries is present on the page that owns it, so no initializer can silently no-op.

All eleven routes were also requested over HTTP and returned 200 with the expected byte counts; both stylesheets and the script serve correctly.

File structure

Path	What lives there
`index.html` → `inventory.html`	The eleven pages — home, four chapters, six modules
`styles/design-system.css`	Tokens and components — the reusable layer
`styles/pages.css`	Page composition and layout
`scripts/app.js`	Shell, state store, animation, page initializers, toast
`ezgif-frame-001…270.jpg`	Existing animation frames, split across the four chapters
`verify.py`	Automated syntax, structure and ID-contract checks
`generate_build_report.py`	This document's generator

Result

Eleven independently addressable pages, one coherent System interface, and a single shared state model. Antigravity's spacing, pills and hairline cards are still the skeleton; the anime theme — near-black panels, violet holograms, cyan signals and a living 270-frame sequence — gives the prototype a cinematic identity it did not have before. Every interactive promise of the original build is kept and, in the case of status derivation, quest economics and the animation system, made more legible.

Fan-made productivity prototype. Solo Leveling is owned by Chugong & Redice Studio. Layout grammar referenced from the Google Antigravity specification; theme and motion inspired by the Solo Leveling anime.